

Chapter 11 .딥러닝_선형회귀/로지스틱회귀

- 케라스 딥러닝 모델 설계
- 딥러닝 선형회귀
- 딥러닝 로지스틱회귀

* 머신러닝/딥러닝 Framework 제공

머신러닝, 딥러닝으로 복잡한 문제를 해결하고 작업하기 위해 다양한 오픈소스 프레임워크 제공
Framework : 응용 프로그램을 개발하기 위한 여러 라이브러리나 모듈 등을 효율적으로 사용할 수 있도록 하나로 묶어 놓은 통합 소프트웨어 개발 패키지

* 딥러닝 프레임워크 비교

	장점	단점
텐서플로우(TensorFlow)	구글이 개발한 오픈소트 라이브러리로 텐서보드를 통해 파라미터 변화 양상이나 DNN의 구조를 알수 있음	- 딥러닝 모델 설계시 기초 레벨부터 직접 작업해야 함으로 초보가 접근하기 어려울 수 있음 - 메모리를 효율적으로 사용하지 못함
케라스(Keras)	배우기 쉽고 모델을 빠르게 구현 최화된 간단하고 일관된 인터페이스 제공	- 오류발생시 케라스 자체 문제인지, 백엔드 문제일지 특정하기 어려움
파이토치(pytorch)	간단하고 직관적으로 학습 가능 속도 대비 빠른 최적화가 가능	- 텐서플로우에 비해 사용자층이 얇음 - 자료 미비

* 케라스(**keras**) : 파이썬으로 작성된 오픈 소스 신경망 라이브러리

- 딥 러닝 모델을 빌드하고 학습하기 위한 **TensowFlow** 상위 수준 **API**
- **keras**가 **Tensorflow**에 통합되어 **tf.keras**로 텐서 플로우 안에서 케라스 사용

* 케라스 딥러닝 모델 설계

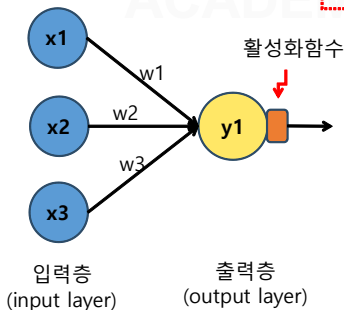
- **Sequential()** : 레이어를 선형(순차모델)으로 연결하기 위한 층 구성
- **Dense()** : 모델층을 설계하고 **add()**를 통해 레이어층 단계적으로
추가
- **compile()** : 모델 학습 전 학습방식에 대한 환경 설정 **loss fuction**
- **fit()** : 모델 학습 **optimizer**
- **evalutate()** , **predict()** : 평가 , 예측

* 전결합층(fully-connected layer) 추가 예시

```
model = Sequential()
```

```
model.add(Dense(1, input_dim=3, activation='relu'))
```

```
# == model.add(Dense(1, input_shape=(3,) , activation='relu'))
```



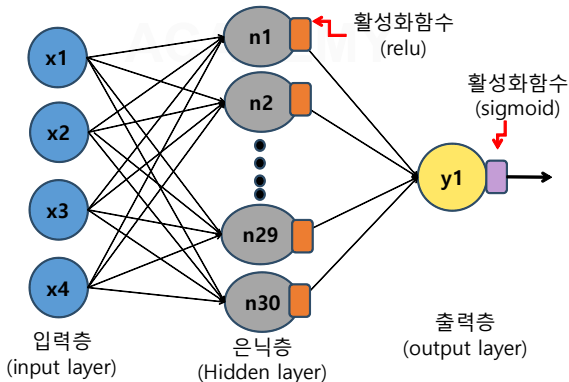
- 첫번째 인자 = 출력 뉴런의 수
- `input_dim` = 입력 뉴런의 수 (입력의 차원)
- `activation` = 활성화 함수
 - linear : 디폴트 값으로 별도 활성화 함수 없는 경우
입력 뉴런과 가중치 연산 결과 그대로 출력(affine)
 - sigmoid : 이진 분류 문제에서 출력층에 주로 사용
 - softmax : 다중 클래스 분류 문제에서 출력층에 주로 사용
 - relu : 은닉층에 주로 사용되는 활성화 함수

* 전결합층(fully-connected layer) 은닉층 추가 예시

```
model = Sequential()
```

```
model.add(Dense(30, input_dim=4, activation='relu') # 입력+은닉층
```

```
model.add(Dense(1, activation='sigmoid') # 출력층
```



- 첫번째 Dense() 의 30인 인자는 출력층의 뉴런 수가 아니라 은닉층의 뉴런수
- 첫번째 Dense()와 달리 두번째 Dense()에 input_dim 인자가 없는 이유는 이미 이전 층의 뉴런 수가 30개임을 알고 있기 때문
- 두번째 Dense()가 마지막 층으로 첫 번째 인자 1은 결국 출력층의 뉴런 수

model.summary() # 모델의 정보를 요약해서 보여줌

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 30)	150
dense_1 (Dense)	(None, 1)	31

Total params: 181

Trainable params: 181

Non-trainable params: 0

model.compile() # 모델의 효과적 학습을 위한 환경설정
보통 손실 함수, 옵티마이저, 메트릭스 정의

model.compile(optimizer='rmsprop', loss='mse', metrics=['accuracy'])

- **optimizer** = 학습 최적화를 위한 옵티마이저 설정 (`sgd`, `rmsprop`, `adam` 등등..)
- **loss** = 학습 과정에서의 손실(비용) 함수 설정
(`mse`, `binary_crossentropy`, `categorical_crossentropy` 등등..)
- **metrics** = 모델을 모니터링하기 위한 지표 선택 (성능 평가 지표)

* 손실(비용)함수 + 출력층 활성화 함수 조합

유형	손실 함수 명	출력층의 활성화 함수 명	
회귀 문제	mean_squared_error (평균 제곱 오차)	-	선형회귀
다중 클래스 분류	categorical_crossentropy (범주형 교차 엔트로피)	소프트맥스 (softmax)	원-핫 인코딩 상태 사용 예) 생선분류, 뉴스분류
다중 클래스 분류	sparse_categorical_crossentropy	소프트맥스 (softmax)	원-핫 인코딩이 된 상태일 필요없이 정수 인코딩 된 상태에서 수행 가능 예) CNN 이미지 분류
이진 분류	binary_crossentropy (이진 교차 엔트로피)	시그모이드 (sigmoid)	1, 0 또는 긍정, 부정 등 2진 분류에 사용, 예) 와인 분류, 영화 리뷰

model.fit() # 모델 학습 (훈련) 시작

모델의 오차 계산 -> 가중치 업데이트 시키는 과정을
반복 학습, 훈련

model.fit(X, Y, batch_size=10, epochs=10, verbose=1) # 모델 훈련

- **첫번째 인자(X)** : 훈련 데이터에 해당
- **두번째 인자(Y)** : 지도학습에서 레이블(타겟) 데이터에 해당
- **batch_size** : 배치 크기, 기본값은 32 , 미니배치경사 하강법을 사용하지 않을 경우 batch_size=None
- **epochs** : 에포크, 1 에포크는 전체 데이터를 모두 한 차례 훈련을 맞춘 상태, 총 훈련 회수 지정'
- **verbose** : 훈련과정 출력 여부,
0 : 훈련과정 미 출력, 1 : 훈련과정 진행막대와 손실등의 지표 출력
2 : 훈련과정 진행막대와 출력을 제외한 매 epoch의 손실, 정확도만 출력
- **validation_data(x_val, y_val)** : 검증데이터 별도 지정, 모델 성능 평가 데이터로만 사용
- **validation_split = 0.2** : 검증데이터 별도 지정이 아니라 훈련 데이터/타겟을 일정 비율 분리하여 검증데이터로 사용, 0.2 지정 시 20% 비율

model.evaluate() # 테스트 데이터를 통해 모델에 대한 정확도 평가

model.evaluate(X_test, Y_test, batch_size=10) # 모델 평가

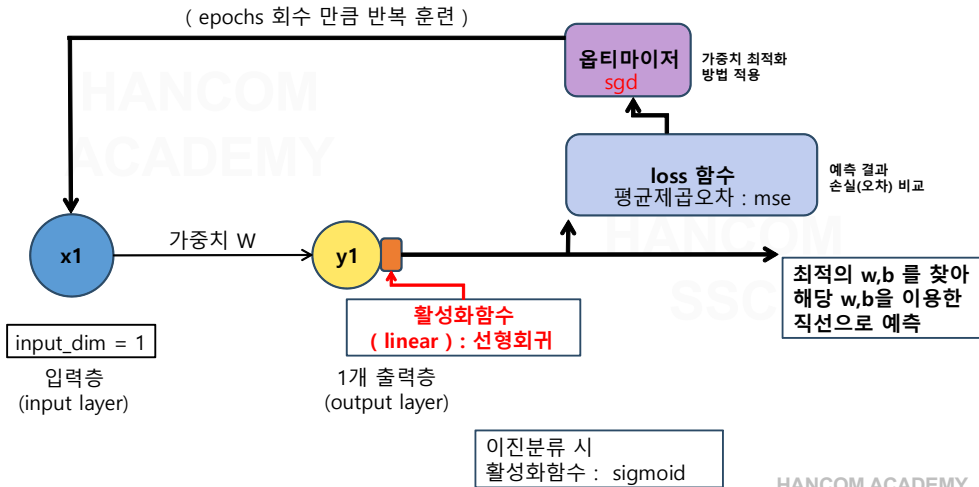
- 첫번째 인자(X_test) : 테스트 데이터에 해당
- 두번째 인자(Y_test) : 지도학습에서 테스트 레이블(타겟)에 해당
- batch_size : 배치 크기

model.predict() # 임의의 데이터 입력에 대한 모델의 예측 출력

model.predict(X_input) # 모델 예측

- X_input : 임의의 예측하고자 하는 데이터

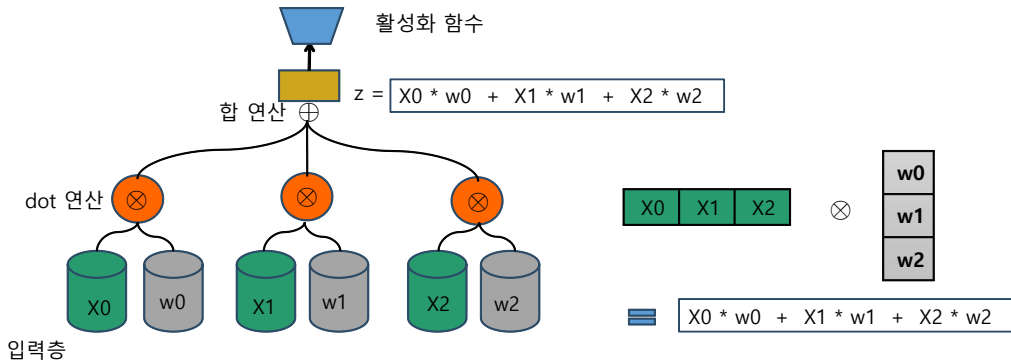
* keras _ 간단한 선형회귀 모델 설계



* 가중치 dot 연산

층별로

입력데이터 와 가중치 곱의 합 (dot 연산) 결과가 활성화 함수에 전달



* keras 선형회귀 모델 예제

tensorflow Gpu 사용 메시지 미출력하게 하는 방법 : 아래 2줄 추가

```
import os
```

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
from tensorflow.keras import optimizers
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
X = np.linspace(0, 10, 10)
```

```
print(X.shape, type(X.shape))
```

(10,) <class 'tuple'>

```
#
```

함수 호출시 * 및 ** 연산자를 사용하여 인수 모음을 각각 별도의 위치 또는 키워드 인수로

unpack할 수 있음

```
Y = X + np.random.randn(*X.shape) # *(10,) ==> 10 으로 unpack
```

```
print(Y)
```

[1.4232874 2.25915571 4.36909246 3.13652478 4.10199597 5.83463168
6.946048 6.52103189 8.95670871 9.14291992]

model = Sequential() # keras 신경망 모델 (순차적으로 층을 추가(쌓는)하는 모델), 순차모델



input_dim : 입력 뉴런의 수 설정

units : 출력(히든) 뉴런 수 설정

활성화함수 설정

activation='linear' : **디폴트 값**으로 별도 활성화 함수 없을 경우
입력 뉴런과 가중치로 계산된 결과값이 그대로 출력

activation='relu' : rectifier 함수, 주로 은닉층에 사용

activation='sigmoid' : 시그모이드 함수, 이진 분류 문제의 출력층에 주로 사용

activation='softmax' : 소프트맥스 함수, 다중 분류 문제의 출력층에 주로 사용

Dense() : 입력 + 출력층 추가

model.add(Dense(input_dim=1, units=1, activation='linear', use_bias=False))

옵티마이저 : 가중치 최적화 방법 설정 (sgd, adam, rmsprop 등)

#sgd = optimizers.SGD(learning_rate=0.01) # 확률적 경사하강, 학습률 0.01

모델 구성후 compile()메서드 호출해서 모델 학습 과정을 설정, 모델 실행(훈련) 전 필요한 설정 결합

model.compile(optimizer='adam', loss='mse') # 옵티마이저(최적화) : 확률적경사하강, 손실함수 : 평균제곱오차

```
weights = model.layers[0].get_weights() # 첫번째 layers 의 가중치
```

```
w = weights[0][0][0]  
print('w begin fit() : ', w)
```

훈련 전 초기 가중치

w begin fit() : -1.4479799

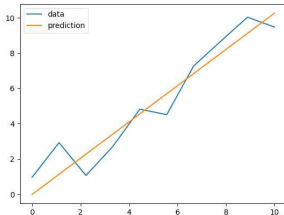
```
model.fit(X, Y, batch_size=1, epochs=1000, verbose=1) # 모델 훈련
```

```
weights = model.layers[0].get_weights() # 훈련 이후 수정된 w 계수 확인
```

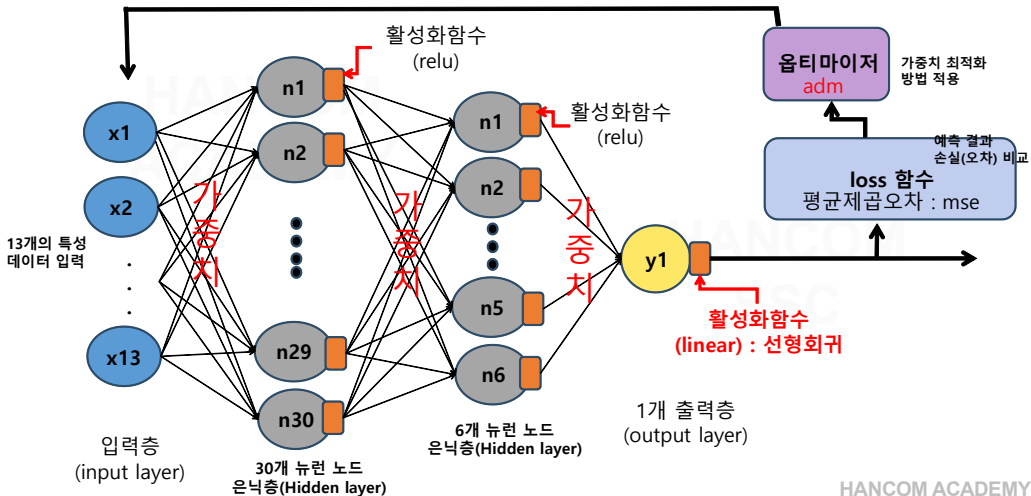
```
w = weights[0][0][0]  
print('w after fit() : ', w)
```

w after fit() : 1.0264648

```
import matplotlib.pyplot as plt  
plt.plot(X,Y, label='data')  
plt.plot(X, w*X, label = 'prediction')  
plt.legend()  
plt.show()
```



* keras _ 선형회귀 모델 활용 보스턴 주택가격 예측



* keras 보스턴 주택가격 예측 예제 (선형회귀)

tensorflow Gpu 사용 메시지 미출력하게 하는 방법 : 아래 2줄 추가

```
import os
```

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
import numpy as np
```

```
import pandas as pd
```

```
Houseinfo = pd.read_csv('BostonHousing.csv')
```

```
print(Houseinfo)
```

```
print(Houseinfo.info())
```

```
x = Houseinfo.iloc[:,0:13]
```

```
y = Houseinfo.iloc[:,13]
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from sklearn.model_selection import train_test_split
```

```
train_x, test_x, train_y, test_y = train_test_split(x, y, test_size=0.3, random_state=42)
```

보스턴 주택가격
데이터셋 로드

CRIM	자치시(town) 별 1인당 범죄율
ZN	25,000 평방피트를 초과하는 거주지역의 비율
INDUS	비소매상업지역이 점유하고 있는 토지의 비율
CHAS	찰스강에 대한 더미변수 (강의 경계에 위치한 경우는 1, 아니면 0)
NOX	10ppm 당 농축 일산화질소
RM	주택 1가구당 평균 방의 개수
AGE	1940년 이전에 건축된 소유주택의 비율
DIS	5개의 보스턴 직업센터까지의 접근성 지수
RAD	방사형 도로까지의 접근성 지수
TAX	10,000 달러 당 재산세율
PTRATIO	자치시(town)별 학생/교사 비율
B	$1000(Bk-0.63)^2$, 여기서 Bk는 자치시별 흑인의 비율을 말함.
LSTAT	모집단의 하위계층의 비율(%)
MEDV	본인 소유의 주택가격(중앙값) (단위: \$1,000)

훈련셋, 테스트셋 분리

* keras 보스턴 주택가격 예측 예제 (선형회귀)

특성데이터 크기가 상이할 경우 정규화 중요!!

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
train_scaled = scaler.fit_transform(train_x)
```

```
# fit() 했음으로 transform만 하면됨
```

```
test_scaled = scaler.transform(test_x)
```

```
model = Sequential()
```

```
model.add(Dense(30, input_dim=13, activation='relu'))
```

```
model.add(Dense(6, activation='relu'))
```

```
model.add(Dense(1,activation='linear')) # activation = 'linear' 생략 가능 , default = 'linear'임
```

```
model.compile(loss='mse', optimizer='adam',metrics=['mse'])
```

회귀모델 경우 성능평가 모니터링 지표로 평균제곱오차 사용

결국 loss 와 동일 함으로 loss만 모니터링 해두 됨! (즉, metrics=['mse'] 생략 가능)

모델 설계 및
컴파일

* keras 보스턴 주택가격 예측 예제 (선형회귀)

```
model.fit(train_scaled, train_y, epochs=200, batch_size=10, verbose=1)
```

모델 훈련

```
pre = model.predict(test_scaled).flatten() # 예측값을 1차원 배열로 펼침
```

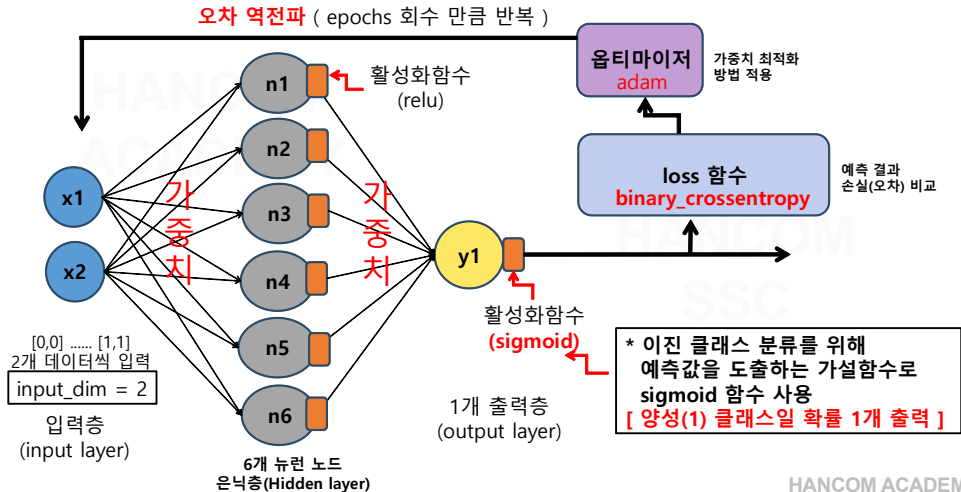
테스트 데이터 예측

```
for i in range(10): # 10개 값만 비교
```

```
    print('실제가격 : {:.3f}, 예상가격 : {:.3f}'.format(test_y.iloc[i], pre[i]))
```

실제가격 : 23.600, 예상가격 : 25.898
실제가격 : 32.400, 예상가격 : 32.700
실제가격 : 13.600, 예상가격 : 15.458
실제가격 : 22.800, 예상가격 : 24.316
실제가격 : 16.100, 예상가격 : 15.050
실제가격 : 20.000, 예상가격 : 21.194
실제가격 : 17.800, 예상가격 : 18.512
실제가격 : 14.000, 예상가격 : 15.081
실제가격 : 19.600, 예상가격 : 25.017
실제가격 : 16.800, 예상가격 : 19.171

* keras _ XOR 문제 설계 (이진분류_로지스틱회귀)



* keras _ XOR 문제 해결_딥러닝 코드

tensorflow Gpu 사용 메시지 미출력하게 하는 방법 : 아래 2줄 추가

```
import os
```

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
x = np.array([[0,0],[0,1],[1,0],[1,1]])
```

```
y = np.array([[0],[1],[1],[0]])
```

```
model = Sequential()
```

```
model.add(Dense(6,input_dim=2, activation='relu'))
```

```
model.add(Dense(1, activation='sigmoid'))
```

```
print(model.summary())
```

입력(2) * 은닉뉴런(6) + 편향(6) = 18

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 6)	18
=====		
dense_1 (Dense)	(None, 1)	7
=====		

Total params: 25

Trainable params: 25

Non-trainable params: 0

* keras _ XOR 문제 해결_딥러닝 코드

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
batch_s = 1
```

```
epoch_c = 1000
```

```
model.fit(x,y,batch_size=batch_s, epochs=epoch_c, verbose=1 ) # 모델 훈련
```

```
print( model.evaluate(x,y,batch_size=batch_s) ) # 모델 평가
```

```
pre = model.predict(np.array([[1,1],[0,1],[1,0]]))
```

출력층
출력값

```
[[0.29430225]  
[0.5508377 ]  
[0.85479707]]
```

양성(1) 클래스일 확률 1
개씩을 출력(예측)

```
print(pre)
```

```
pre = np.where(pre > 0.5, 1, 0 )
```

```
# np.where( 조건식, 조건식 True일 경우 반환값, 조건식 False일 경우 반환값)
```

```
# 두 번째와 세 번째 인수를 생략할 경우 조건식과 일치하는 요소의 인덱스 반환
```

```
print(pre)
```

따라서, 최종 예
측은 0.5 이상일
때 1로 표현하도
록 해야함

```
[[0]  
[1]  
[1]]
```

.....

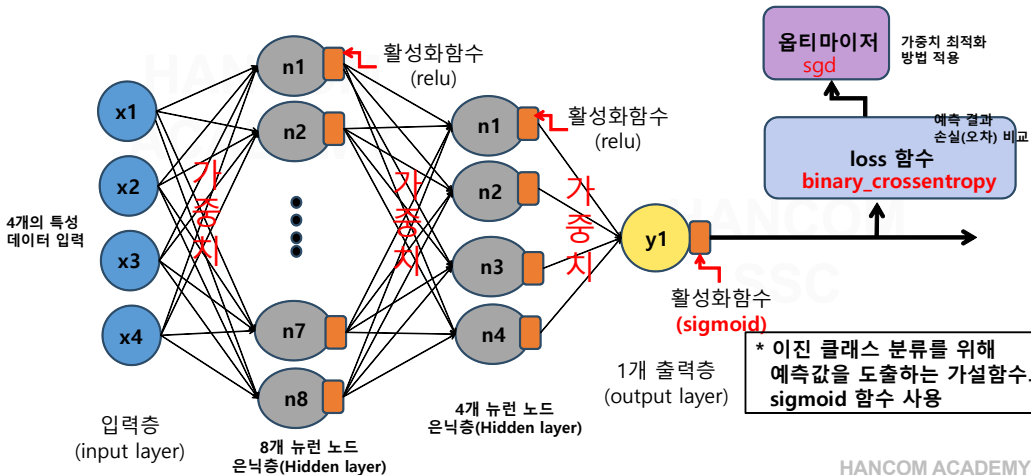
Epoch 1000/1000

4/4 [=====] - 0s 2ms/step - loss: 0.3156 - accuracy: 1.0000

4/4 [=====] - 0s 2ms/step - loss: 0.3996 - accuracy: 1.0000

[0.39955833554267883, 1.0] <= [loss, accuracy]

* keras _ 타이타닉 생존자 분류 (이진분류)



* keras _ 타이타닉 생존자 분류_로지스틱 회귀

tensorflow Gpu 사용 메시지 미출력하게 하는 방법 : 아래 2줄 추가

```
import os
```

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
import numpy as np
```

```
import pandas as pd
```

```
pd.set_option('display.max_rows',20)
```

```
pd.set_option('display.max_columns', 500)
```

```
pd.set_option('display.width',1000)
```

```
titanic_df = pd.read_csv('titanic_passengers.csv')
```

```
print(titanic_df.shape)
```

```
print(titanic_df.head())
```

← 사용할 데이터셋 로드

* keras _ 타이타닉 생존자 분류_로지스틱 회귀

분석에 사용할 특징 데이터 셋 선택

Sex, Age, Pclass 컬럼 데이터셋이 생존에 영향을 주는걸로 가설

Sex 컬럼 'female', 'male' 문자열 데이터를 여성 : 1, 남성 : 0 으로 변경
titanic_df['Sex'] = titanic_df['Sex'].map({'female':1, 'male':0})

결측치 검사

```
print(titanic_df.info())
```

```
print(titanic_df.loc[titanic_df['Age'].isnull(),['Age']]) # 177개 NAN
```

결측치 채우기 : 평균 값

```
titanic_df['Age'].fillna(value=titanic_df['Age'].mean(), inplace=True)
```

```
print(titanic_df.head(10))
```

PClass ==> 1등석, 2등석, 3등석 구분

```
onehot_Pclass = pd.get_dummies(titanic_df['Pclass'], prefix='Class')
```

```
print(onehot_Pclass) # Class_1 Class_2 Class_3 컬럼 생성
```

titanic_df 와 onehot_Pclass 와 결합

```
titanic_df = pd.concat([titanic_df, onehot_Pclass], axis=1)
```

	Class_1	Class_2	Class_3
0	0	0	1
1	1	0	0
2	0	0	1
3	1	0	0
4	0	0	1
...	...	...	...
886	0	1	0
887	1	0	0
888	0	0	1
889	1	0	0
890	0	0	1

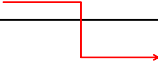
* keras _ 타이타닉 생존자 분류_로지스틱 회귀

```
# 데이터셋 준비
# 입력 데이터 셋 : [Sex, Age, Class_1 , Class_2 ] 컬럼
# 타겟 데이터 셋 : [Survived] 컬럼

titanic_Info = titanic_df[['Sex','Age','Class_1','Class_2']]
titanic_survival = titanic_df['Survived']
print(titanic_Info.head(5))
print(titanic_survival.head(5))

# train dataset / test dataset : 훈련,테스트 데이터셋 분리
from sklearn.model_selection import train_test_split
train_input,test_input,train_target,test_target = \
    train_test_split(titanic_Info, titanic_survival, random_state=42)

print(train_input.shape)
```



(668, 4)

* keras _ 타이타닉 생존자 분류_로지스틱 회귀

특성 데이터 스케일 변환

StandardScaler : 모든 값이 평균 0, 표준편차가 1인 정규분포로 변환

MinMaxScaler : 최소값 0, 최대값 1로 변환

RobustScaler : 중앙값 과 IQR(interquartile range): 25%~75% 사이의 범위 사용해 변환

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
train_scaled = scaler.fit_transform(train_input)
```

```
# fit() 했으므로 transform만 하면됨
```

```
test_scaled = scaler.transform(test_input)
```

```
print(train_scaled) # numpy.ndarray 변환 출력
```

```
print(train_scaled[:,0].std()) # 0번째 열(Sex) 데이터 표준편차 1
```

* keras _ 타이타닉 생존자 분류_로지스틱 회귀

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()
model.add(Dense(input_shape=(4,), units=8, activation='relu'))
model.add(Dense(units=4, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='sgd', loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(train_scaled, train_target, batch_size=64, epochs=1000, verbose=1) # 모델 훈련

score = model.evaluate(test_scaled, test_target) # 모델 테스트 데이터로 평가
print('Test acc : ', score[1]) # 평가 정확도 확인
```

```
model.summary() # 모델 요약 정보
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	40
dense_1 (Dense)	(None, 4)	36
dense_2 (Dense)	(None, 1)	5

Thank You